

Portage du Bytestream Decoder sur Coyote

**Master of Science HES SO in
Engineering**

Profil/Orientation Computer Science

Date of publication: 01.06.2026

Produced by:

Author Abivarman Kandiah

Under the supervision of Andres Upegui Posada, Quentin Berthet



Table of Contents

Declaration of authenticity	3
Acknowledgments	4
Resume (English version)	5
Résumé (version française)	5
1. Context (Résumé Temporaire)	6
2. Introduction	6
3. Contexte technique	7
3.1. Accélération matérielle par FPGA pour le traitement de flux	7
3.2. Frameworks de développement hôte-FPGA	7
3.2.1. XRT	7
3.2.2. Coyote	8
3.3. Le <i>Bytestream Decoder</i> comme algorithme de référence	8
4. Prise en main et évaluation de Coyote	9
4.1. Mise en place de l'environnement	9
4.2. Tests sur U55C — difficultés rencontrées	9
4.3. Tests sur V80 — stabilisation	9
4.4. Retour d'expérience sur Coyote comme framework	9
5. Portage du Bytestream Decoder sur Coyote	9
5.1. Adaptation du wrapper hardware (Design 1)	9
5.2. Adaptation du code hôte	9
6. Analyse des performances et optimisation	10
6.1. Méthodologie de mesure (ILA)	10
6.2. Design 1 — diagnostic du goulot	10
6.3. Design 2 — sortie 128 bits	10
6.4. Montée en fréquence à 400 MHz	10
6.5. Tableau comparatif et bilan	10
7. Discussion : Coyote face à XRT et perspectives smartNIC	11
7.1. Coyote vs XRT — bilan du portage	11
7.2. Pertinence pour des cas d'usage de type smartNIC	11
7.3. Perspectives d'amélioration	11
8. Conclusion	11
Bibliography	12
A Code — Wrapper <code>vfpga_top.svh</code>	14
B Script d'analyse ILA	14





Declaration of authenticity

I hereby affirm that this assignment is my own written work and that I have used no other sources or aids other than those indicated. All passages that have been quoted from publications or paraphrased from these sources are indicated as such. Additionally, any assistance from artificial intelligence tools has been appropriately acknowledged and documented.

Date : 01.06.2026

Name : Abivarman Kandiah



structured-mse-thesis.typ	To be finalized	
HES-SO / Domaine I&A / MSE / Travail de Master	01-06-2026 - Abivarman Kandiah	- 3 / 14 -



Acknowledgments

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.



structured-mse-thesis.typ	To be finalized	
HES-SO / Domaine I&A / MSE / Travail de Master	01-06-2026 - Abivarman Kandiah	- 4 / 14 -



Resume (English version)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Résumé (version française)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.





1. Context (Résumé Temporaire)

L'idée du projet était la prise en main de Coyote, et le test de celui-ci avec des algorithmes déjà implémenté sous XRT. Si possible, aussi confirmer le fonctionnement de celui-ci sur le FPGA V80 qui est nouvellement supporté par Coyote et que nous possédons mais qui était pas vraiment utilisable car ne supporte pas XRT.

J'ai commencé par faire fonctionner Coyote avec le premier design d'exemple sur la U55C. Durant celui-ci j'ai pu rencontré certains problèmes. certains ont été fixés, d'autres je pensais l'avoir fix mais en fait au final il n'est toujours pas stable. En effet, le design se programme bien et le driver semble bien se charger, mais lors de l'exécution de l'host, le programme bloque car n'arrive pas à exécuter le transfert avec le FPGA.

En parallèle, j'ai pu prendre en main en testant le programme qui a été fait par Upegui, qui est le bytestream decoder sous XRT qui est un projet dans le cadre du projet ATLAS. Après avoir testé le bon fonctionnement de celui-ci, j'ai pu commencé à porter le design et le code hôte pour une utilisation sous coyote.

Entre temps, la V80 était officiellement supporté, donc j'ai pu aussi commencé à tester le design d'exemple sous la V80. Avec la V80, j'avais rencontré un problème principal, ou je n'arrivais pas à charger correctement le driver. J'avais un soucis de manque de mémoire pour MSI-X. J'ai pu réglé le problème en ajoutant dans les bootargs, le `pci=realloc`.

Au final, la V80 marche assez bien sous coyote, j'arrive à reproduire correctement à chaque fois contrairement à la U55C.

Le travail du portage côté design est la réécriture du Wrapper, et le fait que coyote a un bus AXI-S de 512 bits, tandis que le design utilise des 32 bits.

2. Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium odio delectus est. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



3. Contexte technique

3.1. Accélération matérielle par FPGA pour le traitement de flux

Les FPGAs sont des matériels utiles à des applications nécessitant des traitements de données à très faible latence, avec des exigences de débit élevées, et une flexibilité d'architecture

Contrairement aux CPUs et aux GPUs, dont l'architecture est figée et le parallélisme borné par le nombre de cœurs ou de threads, un FPGA permet de décrire une architecture sur mesure, dans laquelle chaque étage de traitement s'exécute en parallèle des autres. Cette propriété rend les FPGAs particulièrement adaptés au traitement de flux de données en continu.

Deux cartes ont été utilisées dans le cadre de ce projet. La première, l'Alveo U55C, est une carte d'accélération basée sur un FPGA UltraScale+; La seconde, la V80, est plus récente : elle repose sur l'architecture Versal (NoC matériel intégré, AI Engines), expose une interface PCIe Gen5 x8 et embarque également de la HBM [1].

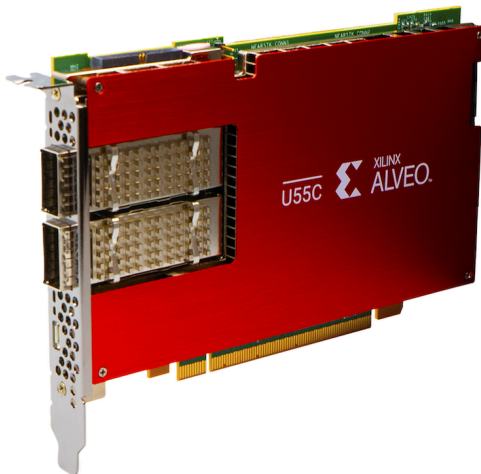


Figure 1: Xilinx Alveo U55C, une carte d'accélération basée sur un FPGA UltraScale+.

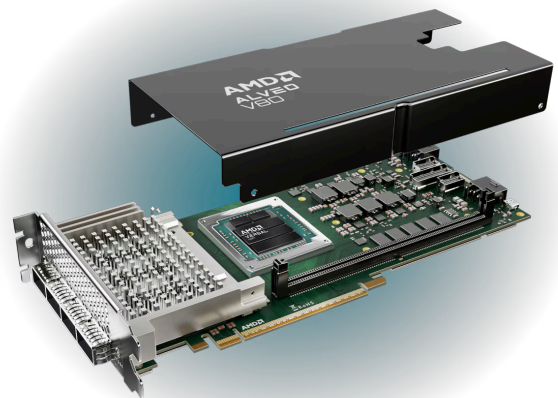


Figure 2: Xilinx Versal V80, une carte d'accélération basée sur un FPGA Versal.

3.2. Frameworks de développement hôte-FPGA

Pour faciliter le développement et le déploiement d'applications sur FPGA, ils existent des frameworks nous fournissant un shell qui est une sorte de système d'exploitation pour FPGA, qui gère les aspects génériques de la communication avec l'hôte (PCIe, DMA, gestion mémoire, etc.) laissant à l'utilisateur la liberté de se concentrer sur la logique applicative. Le shell va généralement permettre à l'utilisateur de charger dynamiquement une application logique sur le FPGA, sans avoir à reprogrammer le FPGA en entier.

Ces frameworks fournissent également une API pour l'hôte, permettant de faciliter les interactions avec le FPGA tel que le chargement de la logique applicative et le transfert de données.

3.2.1. XRT

XRT est le framework officiel d'AMD/Xilinx qui va cibler principalement leurs cartes FPGA [2]. La programmation de la couche logique applicative repose sur des kernels généralement décrits en HLS. La transition des données entre l'hôte et le FPGA suit un schéma comme celui-ci :

XRT bénéficie d'un écosystème mature, intégré à la toolchain Vitis, mais malheureusement ne supporte pas les cartes plus récentes comme la V80.

3.2.2. Coyote

Coyote est un framework open source développé à l'ETH Zürich qui adopte une approche radicalement différente [3]. L'idée centrale est une séparation stricte *shell/role* : le *shell* regroupe toute l'infrastructure générique (contrôleur PCIe, DMA, gestion mémoire, interruptions, hugepages, support multi-locataire et RDMA optionnel) et est livré pré-implémenté ; le *role* correspond à la logique applicative écrite par l'utilisateur, qui dialogue avec le shell via une ou plusieurs interfaces AXI-Stream de 512 bits. Le driver noyau associé est générique et rechargeable sans reflasher le FPGA. Cette architecture cible explicitement les déploiements *dataplane* et *smartNIC*, et le support de cartes récentes — notamment la V80 — y est ajouté plus rapidement que dans XRT. En contrepartie, la documentation est plus parcellaire et la maturité de l'outil reste inférieure à celle de XRT.

Le tableau Table 1 synthétise les principales différences.

	XRT	Coyote
Modèle	Offload batch	Pipeline shell/role
Interface vers le <i>role</i>	Buffers + kernels	AXI-Stream 512 bits
Support V80	Non	Oui
Toolchain requise	Vitis	Vivado seul
Maturité	Élevée	En cours
Cible privilégiée	Calcul accéléré	Dataplane / smartNIC

Table 1: Comparaison synthétique de XRT et Coyote.

3.3. Le *Bytestream Decoder* comme algorithme de référence

L'algorithme retenu pour évaluer Coyote sur un cas réaliste est le *Bytestream Decoder*, un décodeur initialement développé dans le cadre de l'expérience ATLAS au CERN, plus précisément pour la chaîne de lecture du calorimètre à argon liquide (LAr) [4]. Sans entrer dans la physique sous-jacente, il suffit de retenir que les cartes *Front-End Boards* (FEB) du détecteur émettent à chaque collision un flux compressé décrivant l'énergie déposée dans chaque cellule du calorimètre ; le décodeur reconstruit, à la volée, une représentation structurée de ces cellules.

Ce choix est doublement pertinent. D'une part, l'algorithme constitue un cas d'usage représentatif d'une large famille de traitements *dataplane* : décompression et parsing à la volée, consultation de tables (LUT), correction en arithmétique flottante, fusion de plusieurs flux d'entrée. D'autre part, le décodeur existe déjà sous la forme d'un design VHDL validé sous XRT [5] sur Alveo U55C, ce qui en fait un point de comparaison idéal pour mesurer le coût et les bénéfices d'un portage vers Coyote.

Le pipeline VHDL s'organise en plusieurs étages successifs. Un *FEB Parser* extrait les en-têtes et les champs utiles du flux brut, puis des décodeurs spécialisés reconstituent le gain, l'énergie et le temps de chaque cellule. Une table CAM associée à une LUT hachée fournit les coefficients de correction, qui sont appliqués par une unité flottante (FPU) avant qu'un *Output Merger* ne sérialise le résultat dans une FIFO de sortie. En interne, le bus est large de 32 bits ; le module expose deux flux AXI-Stream en entrée (données brutes et corrections initiales) et produit en sortie une structure de quatre mots de 32 bits par cellule (*Gain+ID*, *Energy*,

Time, Quality). Cette largeur de 32 bits, héritée du contexte XRT, jouera un rôle central dans l'analyse de performance présentée au chapitre 5.

4. Prise en main et évaluation de Coyote

4.1. Mise en place de l'environnement

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

4.2. Tests sur U55C — difficultés rencontrées

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

4.3. Tests sur V80 — stabilisation

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

4.4. Retour d'expérience sur Coyote comme framework

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

5. Portage du Bytestream Decoder sur Coyote

5.1. Adaptation du wrapper hardware (Design 1)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

5.2. Adaptation du code hôte

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.

6. Analyse des performances et optimisation

6.1. Méthodologie de mesure (ILA)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.2. Design 1 – diagnostic du goulot

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.3. Design 2 – sortie 128 bits

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

6.4. Montée en fréquence à 400 MHz

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.5. Tableau comparatif et bilan

Design	Fréquence	Sortie	Temps transfert	tready entrée
Design 1	250 MHz	32 bits	3.17 ms	20.9 %
Design 2	250 MHz	128 bits	0.83 ms	88.7 %
Design 2	400 MHz	128 bits	0.82 ms	90.0 %

Table 2: Comparaison des performances des trois configurations testées sur V80.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.



7. Discussion : Coyote face à XRT et perspectives smartNIC

7.1. Coyote vs XRT — bilan du portage

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

7.2. Pertinence pour des cas d'usage de type smartNIC

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

7.3. Perspectives d'amélioration

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

8. Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.



Bibliography

- [1] AMD, “AMD Alveo V80 Compute Accelerator Card – Product Brief.” 2024.
- [2] AMD/Xilinx, “XRT: Xilinx Runtime Library Documentation.” 2024.
- [3] D. Korolija, T. Roscoe, and G. Alonso, “Do OS Abstractions Make Sense on FPGAs?” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/roscoe>
- [4] ATLAS LAr Collaboration, “The ATLAS Liquid Argon Calorimeter: Construction, Integration, Commissioning and Performance,” *Journal of Instrumentation*, vol. 5, p. P9003, 2010.
- [5] A. Upegui Posada, “Bytestream Decoder – design VHDL pour ATLAS LAr sur Alveo U55C.” 2024.
- [6] AMD/Xilinx, “Alveo U55C High Performance Compute Card – Product Brief.” 2022.
- [7] D. Firestone and others, “Azure Accelerated Networking: SmartNICs in the Public Cloud,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2018.



Appendices

A Code — Wrapper vfpga_top.svh	14
B Script d'analyse ILA	14





A Code — Wrapper vfpga_top.svh

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

B Script d'analyse ILA

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

